Based on a thorough review of the uploaded files, here is the list of files that contain explicit security checks, along with a description of the security mechanism in each:

1. **SuperSploit-updated/start.sh**
    - **Security Check:** Root Execution Prevention.
    - **Details:** The script checks if the user's ID is 0 (if [$UID == 0]; then). If the script is executed as the root user, it prints "Do not run as root" and exits immediately to prevent the framework from running with unintended elevated privileges.
2. **SuperSploit-updated/source/core/inputHandler.py**
    - **Security Check:** Binary Integrity Verification (SHA256).
    - **Details:** Includes a checksums class that uses sha256sum to check the integrity of /usr/bin/recon-ng. If the checksum does not match the known safe hash stored in checksums.json, it warns the user before proceeding.
3. **SuperSploit-updated/source/core/reconCore/networkRecon/nmapWrapper.py**
    - **Security Check:** Binary Integrity Verification (SHA256).
    - **Details:** Verifies the nmap binary at /usr/bin/nmap against the .security/checksums.json file before initiating network scans (scan_whole_network and targeted_scan). It warns the user and aborts the scan if the checksum verification fails and the user chooses not to proceed.
4. **SuperSploit-updated/source/core/reconCore/external_tools/bettercap.py**
    - **Security Check:** Binary Integrity Verification (SHA256).
    - **Details:** Checks the integrity of /usr/bin/bettercap using its own local checksums class before executing it as a subprocess.
5. **SuperSploit-updated/source/core/reconCore/external_tools/phoneinfoga.py**
    - **Security Check:** Binary Integrity Verification (SHA256) & Phone Number Length Validation.
    - **Details:** Validates the SHA256 hash of /usr/local/bin/phoneinfoga. Additionally, it performs basic input validation by checking if the supplied phone number is at least 10 digits long (if len(phone_number) < 10:) before proceeding with the scan.
6. **SuperSploit-updated/source/core/reconCore/Bluetooth/blueTooth.py**
    - **Security Check:** File Integrity Verification (SHA256).
    - **Details:** Rather than just checking external system binaries, this file checks the integrity of its own internal framework scripts. It verifies source/core/reconCore/Bluetooth/BlueDucky.py before executing the "ducky" command, and it verifies source/core/reconCore/Bluetooth/blue.sh before executing the "ranger" command.